The Transformer Block

Document 2 of 6 — Building a GPT from Scratch

Document 01 built up the core operation of a transformer: scaled dot-product attention, multi-head attention, and causal masking. But attention alone is not enough to build a deep network. This document covers the remaining components that, together with attention, form the **transformer block** — the repeating unit that gets stacked to build every modern large language model.

We will build four things: residual connections, layer normalization, the position-wise feed-forward network, and then assemble them into a single block.

1 Residual Connections

1.1 The degradation problem

A naive approach to building a more powerful network is to stack more layers. In theory, a deeper network can represent everything a shallower one can (the extra layers could just learn the identity function). In practice, He et al. (2015) showed that this does not happen: beyond a certain depth, adding layers actually *increases* training error. This is not overfitting — it is an optimization failure. The network is harder to train, and gradient-based optimization gets stuck.

The root cause is that gradients must pass through many multiplicative layers during backpropagation. Each layer multiplies the gradient by its Jacobian. If those Jacobians have eigenvalues consistently less than 1, gradients shrink exponentially (vanishing gradients). If greater than 1, they explode. Either way, the optimizer receives a poor signal about how to update early layers.

1.2 The ResNet fix

Instead of asking a sublayer to learn a target function $H(x)$, we ask it to learn the *residual* $F(x) = H(x) - x$:

$$\text{output} = x + F(x)$$

This is a residual connection (also called a skip connection). The sublayer only needs to learn what to *add* to the input, not how to reconstruct it from scratch.

Why does this help gradient flow? Take the gradient of the output with respect to the input:

$$\frac{\partial}{\partial x}\left[x + F(x)\right] = I + \frac{\partial F}{\partial x}$$

The identity matrix I guarantees that even if $\partial F/\partial x$ is very small, the gradient is at least I. In a deep stack of N blocks without residuals, the gradient must pass through N multiplicative factors — it can vanish or explode. With residuals, there is always a "shortcut" path where the gradient is 1. Information from the loss can reach the earliest layers without being distorted by intervening computations.

In code, a residual connection is one line:

Conceptual — illustrating the idea

```
x = x + sublayer(x)
```

Every sublayer in a transformer (attention and feed-forward) is wrapped with a residual connection.

It is useful to think of the tensor flowing through the network as a **residual stream**. Each sublayer reads from the stream, computes a delta, and writes (adds) that delta back. The stream accumulates contributions from every sublayer, and the residual connections ensure that no sublayer can destroy what previous sublayers have written — it can only add to it.

2 Layer Normalization

Deep networks train better when each layer's input has a stable distribution. Normalization layers address this by standardizing activations.

2.1 Why not batch normalization?

Batch normalization (Ioffe & Szegedy, 2015) computes the mean and variance of each feature across the batch dimension. It works well for CNNs, but it is a poor fit for sequence models:

1. **Variable-length sequences.** Different samples in a batch may have different lengths. Computing batch statistics over padded positions mixes real data with padding.
2. **Position-dependent distributions.** Position 1 of every sequence in a batch may have a very different distribution than position 50. Averaging across the batch conflates these.
3. **Inference with batch_size=1.** During generation we process one sequence at a time. Batch statistics over a single example are meaningless.

2.2 Layer normalization

Layer normalization (Ba et al., 2016) sidesteps all of these problems by normalizing across the **feature dimension** for each individual position, independently.

Given an input vector $\mathbf{x}$ of shape $(\ldots, d)$:

1. Compute the mean across features:

$$\mu = \frac{1}{d}\sum_{i=1}^{d} x_i$$

2. Compute the variance across features:

$$\sigma^2 = \frac{1}{d}\sum_{i=1}^{d}(x_i - \mu)^2$$

3. Normalize, scale, and shift:

$$\text{LayerNorm}(\mathbf{x}) = \gamma \odot \frac{\mathbf{x} - \mu}{\sqrt{\sigma^2 + \varepsilon}} + \beta$$

where:

- γ (weight) and β (bias) are learnable parameters of shape $(d,)$.
- ε is a small constant (typically 10^{-5}) to prevent division by zero.

The normalization centers and scales each position's feature vector to zero mean and unit variance. The learnable γ and β then let the model undo this if needed. Each position in the sequence is normalized independently, using only its own d-dimensional feature vector.

2.3 Implementation

Following modern convention (GPT-J, LLaMA), we default to `bias=False`:

This class will go into `src/model.py` in Document 04. For now, understand the structure.

Conceptual — illustrating the idea

```
class LayerNorm(nn.Module):
    def __init__(self, config, eps=1e-5):
        super().__init__()
        self.weight = nn.Parameter(torch.ones(config.n_embd))     # gamma
        self.bias = nn.Parameter(torch.zeros(config.n_embd)) if
            config.bias else None  # beta
        self.eps = eps

    def forward(self, x):
        # mean and variance over the last (feature) dimension
        mu = x.mean(dim=-1, keepdim=True)            # (B, T, 1)
        sigma_sq = x.var(dim=-1, keepdim=True,
                         unbiased=False)              # (B, T, 1)

        # normalize to zero mean, unit variance
        x_normalized = (x - mu) / torch.sqrt(sigma_sq + self.eps)

        # learnable scale and shift
        out = self.weight * x_normalized
        if self.bias is not None:
            out = out + self.bias
        return out
```

Tip: Why `unbiased=False`? The variance formula in the paper divides by d, not $d-1$. PyTorch's `Tensor.var` uses Bessel's correction ($d-1$) by default. We pass `unbiased=False` to get the population variance that matches the LayerNorm definition. `F.layer_norm` does this internally — now you can see it explicitly.

3 Pre-Norm vs Post-Norm

Where you place the LayerNorm relative to the sublayer and the residual connection matters for training stability.

Post-norm (the original "Attention Is All You Need" paper):

$$x = \text{LayerNorm}(x + \text{sublayer}(x))$$

Pre-norm (GPT-2, GPT-3, and most modern transformers):

$$x = x + \text{sublayer}(\text{LayerNorm}(x))$$

3.1 Why pre-norm won

In post-norm, the normalization happens after the residual addition. During early training, the residual x and the sublayer output can have very different magnitudes. Their sum may have large variance, and the normalization must correct for this mismatch at every layer.

In pre-norm, the sublayer always receives a well-conditioned input (zero mean, unit variance) regardless of what earlier layers have done. This decouples the sublayer's optimization from the scale of the residual stream. Training is more stable, especially for deep models.

One consequence of pre-norm: the output of the last transformer block is a residual addition, but there is no normalization applied after that addition. So we need a **final LayerNorm** after the entire stack of blocks.

We use pre-norm throughout BabyGPT.

	Post-norm	Pre-norm
Formula	$x = \mathrm{LN}(x + \mathrm{sublayer}(x))$	$x = x + \mathrm{sublayer}(\mathrm{LN}(x))$
Used by	Original transformer	GPT-2, GPT-3, LLaMA
Final LN needed?	No	Yes
Training stability	Requires careful warmup	More stable out of the box

4 Position-wise Feed-Forward Network

Attention is the inter-position operation: it lets each position gather information from other positions. The feed-forward network (FFN) is the intra-position operation: it processes each position's representation independently, with a larger capacity.

4.1 Architecture

The FFN is a simple two-layer MLP applied to each position:

$$\mathrm{FFN}(\mathbf{x}) = W_2 \cdot \mathrm{GELU}(W_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2$$

where W_1 has shape $(d_{\text{model}}, d_{\text{ff}})$ with $d_{\text{ff}} = 4 \cdot d_{\text{model}}$, and W_2 has shape $(d_{\text{ff}}, d_{\text{model}})$.

Tip: The 4x expansion. The FFN temporarily expands to $4 \times d_{\text{model}}$ dimensions. This gives each position a much wider intermediate representation — more capacity to extract and transform features. The expansion factor 4 is an empirical choice from the original paper that has become standard.

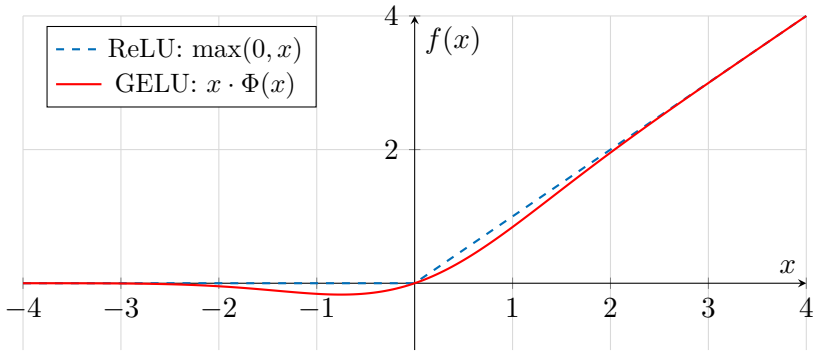
4.2 Activation function

The original transformer uses ReLU: $\max(0, x)$. GPT-2 and most modern language models use GELU:

$$\mathrm{GELU}(x) = x \cdot \Phi(x)$$

where $\Phi(x)$ is the standard Gaussian CDF. Unlike ReLU, which has a hard cutoff at zero, GELU is smooth everywhere — negative inputs are dampened rather than zeroed out.

Tip: What is $\Phi(x)$? The standard Gaussian CDF is: $\Phi(x) = \frac{1}{2}\left[1 + \mathrm{erf}\left(\frac{x}{\sqrt{2}}\right)\right]$ where erf is the error function. $\Phi(x)$ gives the probability that a standard normal random variable is $\leq x$. It rises smoothly from 0 to 1: $\Phi(-\infty) = 0$, $\Phi(0) = 0.5$, $\Phi(\infty) = 1$. Multiplying by x means large positive inputs pass through nearly unchanged ($\Phi \approx 1$), while large negative inputs are suppressed ($\Phi \approx 0$). In practice, PyTorch's `F.gelu` uses a fast tanh-based approximation: $\mathrm{GELU}(x) \approx 0.5\,x\left[1 + \tanh\left(\sqrt{2/\pi}\,(x + 0.044715\,x^3)\right)\right]$.



4.3 Why "position-wise"

The same weights W_1 and W_2 are shared across all positions in the sequence. Each position is processed independently — there is no information flow between positions inside the FFN. It is equivalent to a 1×1 convolution in CNN terminology.

Tip: Shared weights, independent application. If X has shape (T, d_{model}), then XW_1 is $(T, d_{\text{model}}) \times (d_{\text{model}}, d_{\text{ff}}) = (T, d_{\text{ff}})$. Each row of X (one position) is multiplied by the same W_1. In PyTorch, `nn.Linear` broadcasts the matmul across the T dimension automatically — no loop over positions, and no separate FFN per position. This is why it is called "position-wise": one FFN, applied to each position the same way.

4.4 Implementation

This class will go into `src/model.py` in Document 04. For now, understand the structure.

Conceptual — illustrating the idea

```
class FeedForward(nn.Module):
    def __init__(self, config):
        super().__init__()
        self.c_fc = nn.Linear(config.n_embd, 4 * config.n_embd,
            bias=config.bias)
        self.c_proj = nn.Linear(4 * config.n_embd, config.n_embd,
            bias=config.bias)
        self.dropout = nn.Dropout(config.dropout)

    def forward(self, x):
        x = F.gelu(self.c_fc(x))
        return self.dropout(self.c_proj(x))
```

5 Assembling the Transformer Block

A single transformer block combines everything:

1. Attention sublayer with residual and pre-norm: $x = x + \text{CausalSelfAttention}(\text{LayerNorm}(x))$
2. FFN sublayer with residual and pre-norm: $x = x + \text{FeedForward}(\text{LayerNorm}(x))$

The two sublayers serve complementary roles:

- **Attention** handles inter-position communication.
- **FFN** handles intra-position computation.

5.1 Implementation

This class will go into `src/model.py` in Document 04. For now, understand the structure.

Conceptual — illustrating the idea

```
class TransformerBlock(nn.Module):
    def __init__(self, config):
        super().__init__()
        self.ln_1 = LayerNorm(config)
        self.attn = CausalSelfAttention(config)
        self.ln_2 = LayerNorm(config)
        self.ffn = FeedForward(config)

    def forward(self, x):
        x = x + self.attn(self.ln_1(x))
        x = x + self.ffn(self.ln_2(x))
        return x
```

6 Stacking Blocks

A transformer model is N identical blocks applied in sequence. In our BabyGPT, $N = 6$. Each block has its own parameters (not shared).

After the last block, a final LayerNorm is applied:

Preview of the forward pass in `BabyGPT`, which we build in Document 04.

Conceptual — illustrating the idea

```
x = tok_emb + pos_emb
for block in self.blocks:
    x = block(x)
x = self.final_ln(x)
```

6.1 Parameter count

With our config ($d_{\text{model}} = 384$, $d_{\text{ff}} = 1536$, $h = 6$, bias = False):

- Attention: $3 \times (384 \times 384) + 384 \times 384 = 589{,}824$
- FFN: $384 \times 1536 + 1536 \times 384 = 1{,}179{,}648$
- LayerNorm: $2 \times 384 = 768$
- Total per block: 1,770,240 ($\sim$1.77M parameters)
- 6 blocks: $\sim$10.6M parameters

The FFN accounts for about two-thirds of each block's parameters.

7 Looking Ahead

We now have the complete building block. In Document 03, we look at the full encoder-decoder architecture from the original paper. In Document 04, we simplify to the decoder-only GPT architecture we implement.